

## Day 2: Variables & Data Types

- Primitive data types
- Reference data types
- Type casting
- Constants
- Naming conventions

Programs:

- Swap numbers
- Calculate simple interest
- Temperature conversion

### Imagine First...

Suppose nuvvu oka supermarket ki vellavu.

Akada different boxes untayi.

- Rice Box
- Sugar Box
- Oil Bottle
- Water Bottle

Question:

**Oil bottle lo rice pettagalama?**

No.

**Water bottle lo sugar pettagalama?**

Possible but correct kaadu.

Because every container has a purpose.

Computer memory kuda exactly alage work chestundi.

Java says

"Before storing data, tell me what type of data it is."

Anduke Variables and Data Types vachayi.

---

## What is a Variable?

Real life example:

Suppose teacher asks

"What is your name?"

Nuvvu chepthav

Harika

Computer ela remember chestundi?

It stores inside memory.

Memory lo oka small place create chestundi.

Aa place ki name istundi.

Example

Name : ALEX

Aa name place ni Java lo

### **Variable**

antaru.

Definition:

Variable is a named memory location used to store data.

Simple words:

Variable = Memory box.

---

### **Memory Visualization**

Without variable

Memory

123456

987654

Hello

98.5

Computer ki ardham kaadu.

With variable

name = Alex

age = 20

marks = 95

salary = 50000

Now everything is organized.

---

## **Syntax**

`datatype variableName = value;`

Example

`int age = 20;`

Break it.

`int`

means

"I want integer storage."

`age`

Variable name.

`=`

Assignment operator.

Means

Store right side value into left side variable.

`20`

Actual value.

---

## **Memory Representation**

`int age = 20;`

Memory

age

-----

| 20 |

-----

Whenever you write

`System.out.println(age);`

Java goes to memory

Finds age

Prints

20

---

## **Why Variables?**

Without variable

`System.out.println(20);`

If age changes?

Again edit.

With variable

`age = 21;`

Automatically updated.

Variables make programs dynamic.

---

## **Rules for Variable Names**

Java follows rules.

---

### **Rule 1**

Can start with

Letter

age

Correct

---

Underscore

\_age

Correct

---

Dollar

\$salary

Correct

(Not recommended.)

---

Cannot start with number

Wrong

1age

✗

---

Correct

age1

✓

---

## Rule 2

No spaces

Wrong

student age(space between student and age)

Correct

studentAge

---

## Rule 3

No special symbols

Wrong

age@

Wrong

salary#

---

#### **Rule 4**

Keywords cannot be variable names.

Wrong

```
int int = 10;
```

Because

int

already Java keyword.

---

#### **Variable Naming Conventions**

Convention ante

Rule kaadu

Best practice.

Java developers follow this.

Example

studentName

totalMarks

bankBalance

employeeSalary

#### **Camel Case.**

First word small.

Second word starts with capital.

Example

studentAge

Not

StudentAge

Not

student\_age

CamelCase is standard.

---

## **Primitive Data Types**

Now biggest topic.

Java stores different kinds of data.

Question

Can age be stored as String?

Yes.

But waste.

Why?

Because String occupies more memory.

Java provides special storage types.

These are

Primitive Data Types.

There are 8.

---

## **Primitive Data Types**

Byte

Short

int

long

float

double

char

boolean

Let's understand each deeply.

---

## 1 Byte

Size

1 Byte

Stores

Very small integers.

Range

-128 to 127

Example

byte age = 25;

Memory

age

25

When to use?

Very small values.

Example

Student section number.

---

## Why range only 127?

Computer stores in binary.

1 byte

=

8 bits.

Maximum value becomes

127.

No need to memorize binary now.

Later learn.

---



## **2 Short**

Size

2 bytes

Stores

Larger integers.

Range

Approx

-32000 to +32000

Example

short salary = 25000;

---

## **3 Int**

Most commonly used.

Size

4 bytes

Range

Very large.

Around

-2 billion

to

+2 billion

Example

int marks = 98;

Use for

Age

Marks

Population

Students

etc.

---

## Why int is default?

Because

Balance between

Memory

Speed

Performance

---

## 4 Long

Very very large integers.

Size

8 bytes

Example

```
long population = 8000000000L;
```

Notice

L

Without L

Java thinks int.

---

## 5 Float

Stores decimal numbers.

Size

4 bytes

Example

```
float price = 25.5f;
```

Need

f

Otherwise Java assumes double.

---

## 6 Double

Most used decimal type.

Size

8 bytes

Example

```
double pi = 3.1415926535;
```

Higher precision.

Default decimal type.

---

## Float vs Double

Float

Less memory

Less accuracy

Double

More memory

More accuracy

Nowadays

Mostly use double.

---

## 7 Char

Stores only one character.

Example

```
char grade = 'A';
```

Important

Single quotes.

'A'

Correct

"A"

Wrong

Because

Double quotes means String.

---

## ASCII Concept

Actually

'A'

internally stored as

65.

'B'

66.

Later Unicode learn.

---

## 8 Boolean

Stores

Only

true

false

Example

boolean passed = true;

Used in

if conditions.

---

## Primitive Summary

| Data Type Stores |                 | Example |
|------------------|-----------------|---------|
| byte             | Small integers  | 20      |
| short            | Medium integers | 25000   |
| int              | Normal integers | 100     |

| Data Type Stores |                  | Example      |
|------------------|------------------|--------------|
| long             | Huge integers    | 90000000000L |
| float            | Decimal          | 2.5f         |
| double           | Accurate decimal | 3.14159      |
| char             | Single character | 'A'          |
| boolean          | True/False       | true         |

---

## Reference Data Types

Now important concept.

Primitive stores value directly.

Reference stores address.

Imagine.

You have a house.

Instead of storing entire house,

you store

House Address.

Exactly Reference Type.

---

Example

String name = "Bob";

Memory

Stack

name

↓

Heap

Bob

Variable contains address.

Not actual object.

---

Examples

String

Array

Scanner

Class Objects

All are Reference Types.

---

## **Difference**

Primitive

```
int age = 20;
```

Stores

20

Directly.

Reference

```
String name = "Bob";
```

Stores

Address.

---

## **Type Casting**

Meaning

Changing one datatype into another datatype.

Real life

Water bottle

↓

Bucket

Container changes.

Water same.

---

Types

1 Implicit

2 Explicit

---

## **1 Implicit Casting**

Small

↓

Big

Automatic.

Example

```
int a = 10;
```

```
double b = a;
```

Memory

a

10

↓

b

10.0

Java automatically converts.

Safe conversion.

---

## 2 Explicit Casting

Big

↓

Small

Manual.

Example

```
double x = 25.9;
```

```
int y = (int)x;
```

Output

25

Decimal removed.

---

Why manual?

Because

Data loss possible.

---

Example

double

99.99

↓

int

99



0.99 lost.

---

## **Constants**

Suppose

PI value

Always

3.14159

Should not change.

Use

final

Example

```
final double PI = 3.14159;
```

Later

```
PI = 4;
```

Error.

Because constant.

---

## **Naming Convention**

Constants

Always

Uppercase.

Example

MAX\_VALUE

PI

MIN\_MARKS

---

## **Difference**

Variable

Can change.

age = 20

age = 21

Constant

Cannot change.

---

## **Program 1**

Swap Two Numbers

Example

Before

a = 10

b = 20

After

a = 20

b = 10

Using temporary variable:

```
public class SwapNumbers {  
    public static void main(String[] args) {  
        int a = 10;  
        int b = 20;  
  
        System.out.println("Before Swap:");  
        System.out.println("a = " + a);  
        System.out.println("b = " + b);
```

```
int temp = a; // Store a temporarily
a = b;        // Copy b into a
b = temp;     // Copy original a into b

System.out.println("After Swap:");
System.out.println("a = " + a);
System.out.println("b = " + b);
}
}
```

### **Dry Run**

Initial:

a = 10

b = 20

Step 1:

temp = a

temp = 10

Step 2:

a = b

a = 20

Step 3:

b = temp

b = 10

Final:

a = 20

b = 10

---

### **Program 2**

Simple Interest

Formula:

$$SI = (P \times R \times T) / 100$$

Where:

- **P** = Principal Amount
- **R** = Rate of Interest
- **T** = Time (years)

```
public class SimpleInterest {  
    public static void main(String[] args) {  
        double principal = 10000;  
        double rate = 5;  
        double time = 2;  
  
        double simpleInterest = (principal * rate * time) / 100;  
  
        System.out.println("Simple Interest = " + simpleInterest);  
    }  
}
```

### **Dry Run**

principal = 10000

rate = 5

time = 2

$$10000 \times 5 \times 2 = 100000$$

$$100000 / 100 = 1000$$

Output:

Simple Interest = 1000.0

---

### **Program 3**

Temperature Conversion (Celsius → Fahrenheit)

Formula:

$$F = (C \times 9/5) + 32$$

```
public class TemperatureConversion {  
    public static void main(String[] args) {  
        double celsius = 30;  
  
        double fahrenheit = (celsius * 9 / 5) + 32;  
  
        System.out.println("Temperature in Fahrenheit = " + fahrenheit);  
    }  
}
```

### Dry Run

celsius = 30

$$30 \times 9 = 270$$

$$270 / 5 = 54$$

$$54 + 32 = 86$$

Output:

Temperature in Fahrenheit = 86.0

---

### Common Beginner Mistakes

- ✗ Forgetting ; at the end of a statement.
- ✗ Using = instead of == in conditions (you'll learn == on Day 4).
- ✗ Writing:

```
char ch = "A";
```

Correct:

```
char ch = 'A';
```

- ✗ Forgetting L with large long values:

```
long n = 9000000000L;
```

- ✗ Forgetting f with float values:

```
float x = 3.5f;
```

✗ Trying to change a final constant:

```
final int MAX = 100;
```

```
MAX = 200; // Error
```

---

## Interview Questions (Very Important)

1. What is a variable?

A **variable** is a named memory location used to store data that can change during program execution.

2. Why do we use variables?

Variables are used to store, access, and modify data during program execution.

3. What is the syntax for declaring a variable?

```
datatype variableName = value;
```

4. What are primitive data types?

Primitive data types are **built-in data types**(ante already existed java language rasinappudu) provided by Java that store simple values directly in memory.

5. Name all 8 primitive data types.

Byte ,short ,int, long, float, double, char, boolean

6. Difference between primitive and reference data types.

| Primitive           | Reference              |
|---------------------|------------------------|
| Stores actual value | Stores object address  |
| Fixed size          | Size depends on object |
| Cannot store null   | Can store null         |
| No methods          | Has methods            |

## Detailed Explanation

Primitive:

```
int age = 20;
```

Memory:

age

20

Reference:

String name = "Alex";

Memory:

name -----> "Alex"

name stores the address of the String object.

Examples of reference types:

- String
- Array
- Scanner
- Objects

---

## 7. Difference Between float and double

### Short Answer

| float          | double           |
|----------------|------------------|
| 4 bytes        | 8 bytes          |
| Less precision | More precision   |
| Uses f suffix  | No suffix needed |

### Detailed Explanation

Example:

float price = 25.5f;

Needs f.

Example:

double pi = 3.1415926535;

No suffix needed.

Use:

- float → when memory is important

- double → when accuracy is important

Nowadays, developers mostly use **double**.

---

## 8. Difference Between char and String

### Short Answer

| <b>char</b>          | <b>String</b>              |
|----------------------|----------------------------|
| Stores one character | Stores multiple characters |
| Uses single quotes   | Uses double quotes         |
| Primitive            | Reference                  |

### Detailed Explanation

Character:

```
char grade = 'A';
```

String:

```
String name = "Harika";
```

Wrong:

```
char c = "A";
```

Correct:

```
char c = 'A';
```

---

## 9. What is Type Casting?

### Short Answer

Type casting is the process of converting one data type into another.

### Detailed Explanation

Example:

```
int a = 10;
```

```
double b = a;
```

Output:

10.0



Another example:

```
double x = 25.9;
```

```
int y = (int)x;
```

Output:

25

The decimal part is removed.

---

## 10. Difference Between Implicit and Explicit Casting

### Short Answer

| Implicit      | Explicit           |
|---------------|--------------------|
| Automatic     | Manual             |
| Small → Large | Large → Small      |
| No data loss  | Data loss possible |

### Detailed Explanation

Implicit:

```
int a = 10;
```

```
double b = a;
```

Java converts automatically.

Explicit:

```
double x = 25.9;
```

```
int y = (int)x;
```

You manually tell Java to convert.

---

## 11. What is a Constant?

### Short Answer

A constant is a variable whose value cannot be changed after initialization.

### Detailed Explanation

Example:

```
final double PI = 3.14159;
```

Later:

```
PI = 4;
```

Produces a compile-time error because PI is constant.

Examples:

- PI
  - MAX\_VALUE
  - PASS\_MARKS
- 

## 12. Why do we use the final Keyword?

### Short Answer

The final keyword is used to make a variable constant so its value cannot be modified.

### Detailed Explanation

Example:

```
final int MAX = 100;
```

Valid:

```
System.out.println(MAX);
```

Invalid:

```
MAX = 200;
```

Java gives an error because MAX is final.

---

## 13. What is Camel Case Naming Convention?

### Short Answer

Camel Case is a naming style where the first word starts with a lowercase letter and each subsequent word starts with an uppercase letter.

### Detailed Explanation

Correct:

```
studentName
```

```
totalMarks
```

employeeSalary

Wrong:

Student\_name

student\_name

StudentName

Java developers commonly use camelCase for variables and methods.

---

## 14. Why is int the Most Commonly Used Integer Type?

### Short Answer

Because int provides a good balance between memory usage, performance, and range.

### Detailed Explanation

int:

- Uses 4 bytes
- Stores values from approximately **-2.1 billion to +2.1 billion**
- Fast on most computers
- Suitable for most calculations

Example:

```
int age = 20;
```

```
int marks = 95;
```

```
int salary = 50000;
```

Most integer values fit comfortably within the int range, so Java programmers usually choose int unless they specifically need a smaller or larger type.

---

## 15. Why does long use the L Suffix?

### Short Answer

Because Java treats integer literals as int by default. The L suffix tells Java that the value is a long.

### Detailed Explanation

Wrong:

```
long population = 9000000000;
```

This gives a compile-time error because 9000000000 is larger than the maximum int value.

Correct:

```
long population = 9000000000L;
```

Here, L tells Java to treat the number as a long.

**Tip:** Use an uppercase L (9000000000L) because lowercase l looks similar to the digit 1.

---